DAW · PROYECTO INTERMODULAR

MEMORIA TÉCNICA · ENTREGAS 1 A 3

Tupper
planificación semanal inteligente de comidas

Introducción, objetivos, elección de tecnologías, planificación, diseño técnico (entidad-relación, arquitectura, flujo funcional, casos de uso, esquema relacional, secuencia, despliegue y Gantt), mockups y control de versiones del proyecto de desarrollo de aplicaciones web.

Tupper es el nombre de la aplicación; **Recetario** es el nombre de trabajo del proyecto durante su desarrollo.

Roberto de la Casa Sotillo
Ciclo Formativo de Grado Superior · DAW

Actualizada el 9 de noviembre de 2026
Entregas 1 a 3 · Curso 2026/2027
github.com/metatron18es/recetario

Índice

ÍNDICE GENERAL

01 Introducción 1
02 Objetivos del proyecto 2
03 Objetivos personales 2
04 Planificación 3
05 Roadmap y diagrama de Gantt 5
06 Elección de tecnologías 6
07 Diagrama de casos de uso 7
08 Diagrama de flujo funcional 8
09 Diagrama de arquitectura 9
10 Diagrama entidad-relación 10
11 Esquema relacional 11
12 Secuencia: autenticación 12
13 Secuencia: generación del calendario 13
14 Mockups de las pantallas principales 14
15 Control de versiones y estado del esqueleto 16
16 Diagrama de despliegue 17

ÍNDICE DE FIGURAS Y TABLAS

Figura 1 Cronograma del proyecto (cinco fases) 3
Tabla 1 Fases, periodos y contenido del proyecto 3
Figura 2 Tablero Kanban del proyecto 4
Figura 3 Roadmap de épicas 5
Figura 4 Diagrama de Gantt 5
Figura 5 Diagrama de casos de uso 7
Figura 6 Diagrama de flujo funcional 8
Figura 7 Diagrama de arquitectura 9
Figura 8 Diagrama entidad-relación 10
Figura 9 Esquema relacional 11
Figura 10 Diagrama de secuencia: autenticación 12
Figura 11 Diagrama de secuencia: generación del calendario 13
Figura 12 Wireframes de las pantallas principales 14
Figura 13 Wireframe del formulario de preferencias 15
Figura 14 Diagrama de despliegue 17

01 Introducción

> Recetario es una aplicación web en la que los usuarios comparten sus recetas y que genera automáticamente el calendario semanal de comidas de cada uno, adaptado a sus alergias, su dieta y su objetivo calórico.

Organizar una alimentación saludable y variada a lo largo de la semana es una tarea que requiere tiempo y conocimientos nutricionales de los que no todo el mundo dispone. Es habitual recurrir siempre a las mismas recetas por comodidad, o renunciar a cocinar de forma equilibrada por falta de planificación.

Recetario permite a cualquier usuario publicar sus recetas de cocina, que quedan disponibles para toda la comunidad, y generar automáticamente un calendario semanal de comidas (desayuno, almuerzo, comida, merienda y cena) adaptado a sus necesidades: sus alergias e intolerancias, el tipo de dieta que sigue y su objetivo calórico diario.

Las recetas son públicas: cualquier usuario puede consultarlas y se guarda quién las ha publicado. Solo su autor puede editarlas y solo el administrador puede borrarlas; si se borra una receta, desaparece para todos y las celdas de calendario que la usaban quedan vacías. Si un usuario se da de baja, sus recetas se conservan. Los ingredientes nuevos que propone un usuario los valida el administrador antes de que los utilice el motor de reglas.

A diferencia de un recetario tradicional, que se limita a almacenar y mostrar recetas, Recetario incorpora un **motor de reglas nutricionales propio** que decide automáticamente qué recetas encajan en el plan semanal de cada usuario, evitando ingredientes no permitidos y ajustando el aporte calórico total, sin que el usuario tenga que hacer ese cálculo manualmente.

Arquitectura. Frontend en Vue 3 + TypeScript que consume una API REST en Node.js/Express, base de datos relacional PostgreSQL gestionada mediante Prisma, y despliegue en producción sobre Vercel y Neon.

02 Objetivos del proyecto

Este proyecto tiene como finalidad desarrollar una aplicación web completa de gestión de recetas con generación automática de calendarios de comidas saludables. Para ello se persiguen los siguientes objetivos:

- Diseñar y desarrollar una aplicación web completa de gestión de recetas de cocina.
- Implementar un sistema de autenticación seguro basado en JWT.
- Modelar una base de datos relacional que almacene recetas, usuarios, preferencias nutricionales y calendarios de comidas.
- Desarrollar un motor de reglas que genere automáticamente calendarios semanales de comidas ajustados a las restricciones y objetivos nutricionales de cada usuario.
- Diseñar una interfaz de usuario clara y accesible para gestionar recetas y visualizar el calendario semanal.
- Desplegar la aplicación en un entorno de producción real, con un pipeline de integración y despliegue continuo (CI/CD).
- Garantizar la calidad y seguridad del software mediante pruebas automatizadas y buenas prácticas (OWASP).

03 Objetivos personales

CONSOLIDAR

- Afianzar los conocimientos adquiridos durante el ciclo en frontend, backend y bases de datos, integrándolos en un proyecto real de principio a fin.

PROFUNDIZAR

- Profundizar en el diseño de un motor de reglas / lógica de negocio no trivial, más allá de un CRUD básico.

EXPLORAR

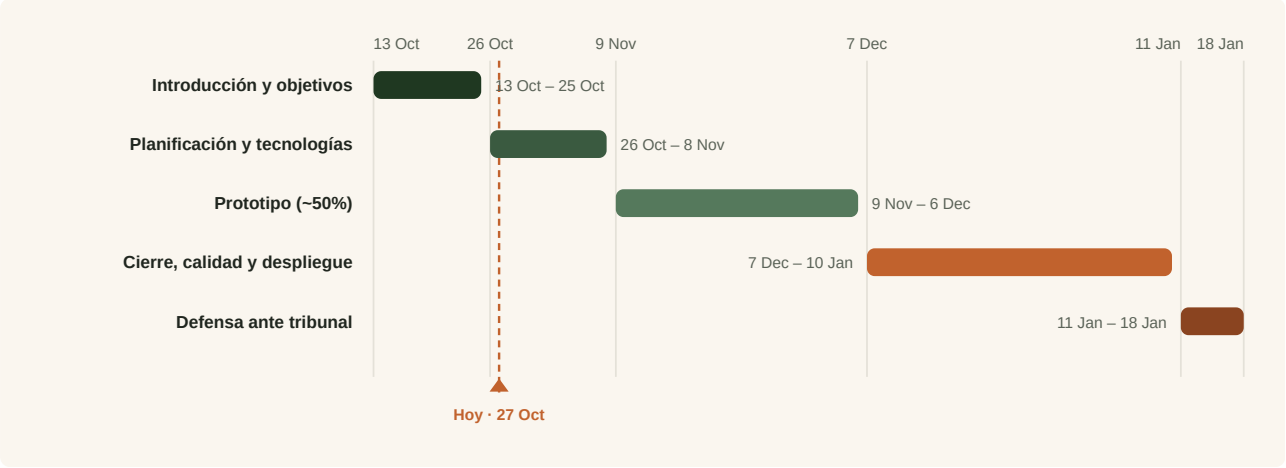
- Aprender a desplegar una aplicación en un entorno serverless (Vercel + Neon), algo no visto en profundidad durante el ciclo.

RESPONSABILIDAD CON IA

- Usar herramientas de IA de forma responsable como apoyo al desarrollo, entendiendo y pudiendo defender cada parte del código entregado.

04 Planificación

El proyecto se organiza en cinco fases con fecha de entrega fija marcada por el tutor, gestionadas mediante un tablero Kanban en GitHub (no sprints cerrados, al ser un desarrollo individual sin equipo con el que coordinar iteraciones).



CRONOGRAMA DEL PROYECTO · CINCO FASES · OCTUBRE 2026 – ENERO 2027

FASE	PERIODO	DURACIÓN	QUÉ INCLUYE
Introducción y objetivos	13–25 oct	13 días	Backlog de historias de usuario, elección del stack tecnológico, apartados 1-2 de la memoria
Planificación y tecnologías	26 oct – 8 nov	14 días	Diagrama entidad-relación, diagrama de arquitectura, diagrama de flujo funcional, cronograma, mockups, esqueleto desplegado
Prototipo (~50%)	9 nov – 6 dic	28 días	Autenticación, CRUD de recetas, preferencias nutricionales, motor de reglas, diseño del protocolo de pruebas
Cierre, calidad y despliegue	7 dic – 10 ene	35 días	Calendario semanal editable, tests automatizados, despliegue final en producción, resto de apartados de la memoria
Defensa ante tribunal	11–18 ene	8 días	Ensayo cronometrado de la exposición oral

El desglose completo en tareas, con las etiquetas de epic y prioridad, está en el tablero Kanban del proyecto y en los hitos del repositorio, donde cada fase es un hito con su fecha límite y las tareas (issues) asignadas.

04 Planificación (continuación)

El seguimiento del trabajo se lleva en un proyecto de GitHub Projects, con un tablero Kanban de cinco columnas. Cada historia de usuario y cada tarea es una issue etiquetada con su épica y su prioridad (MoSCoW) y asignada al hito de su entrega.



TABLERO KANBAN DEL PROYECTO EN GITHUB PROJECTS · ESTADO A 9 DE NOVIEMBRE DE 2026

Columnas — Backlog, Lista para hacer, En progreso, En revisión/test y Hecho.

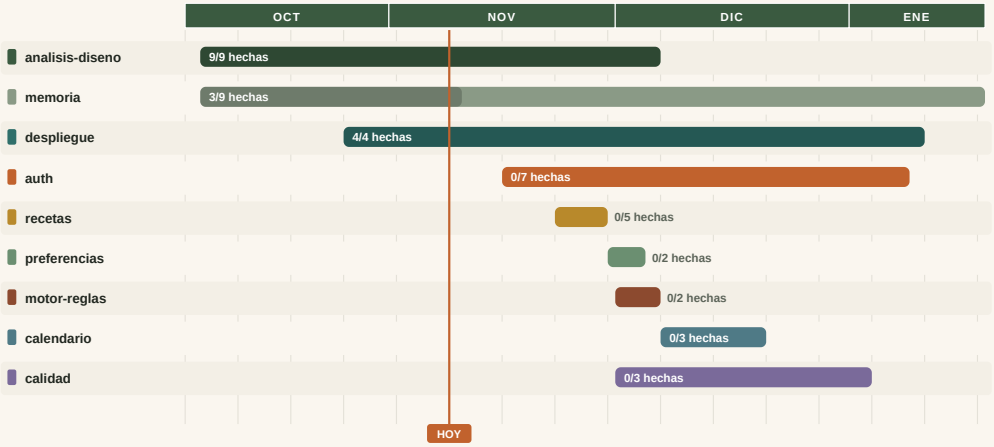
Épicas — nueve: auth, recetas, preferencias, motor de reglas, calendario, calidad, despliegue, análisis y diseño, y memoria.

Hitos — uno por entrega (1, 2, 3 y final) más la defensa ante tribunal; 44 issues en total.

05 Roadmap y diagrama de Gantt

> El roadmap muestra el periodo de cada épica según las fechas del proyecto de GitHub; el Gantt detalla las cinco fases con el desglose del prototipo y del cierre (reparto semanal previsto).

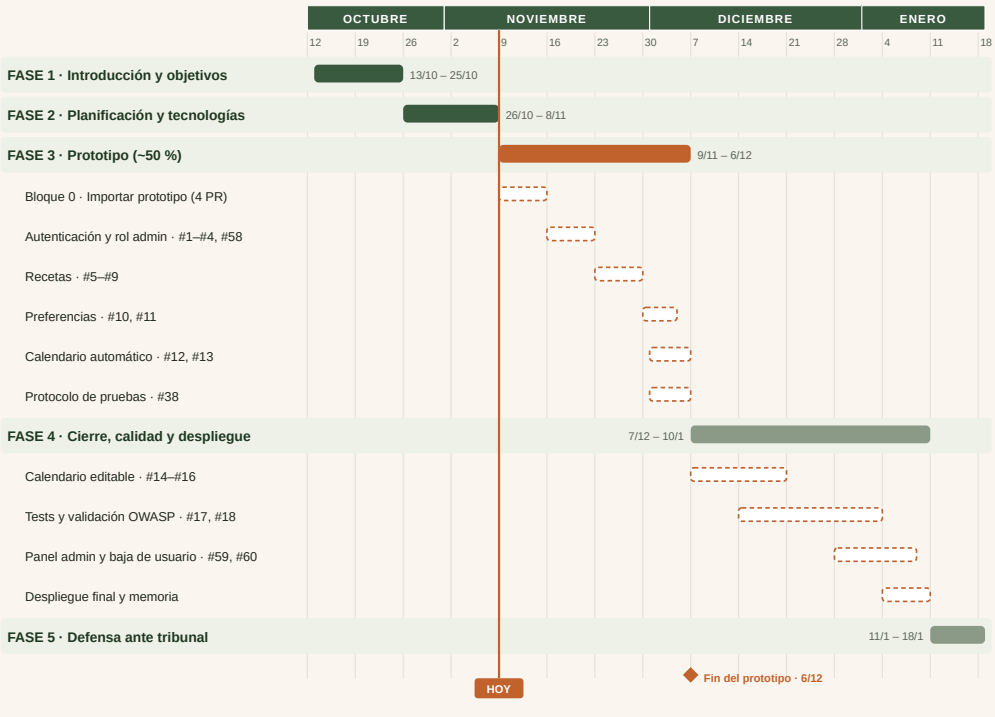
ROADMAP · ÉPICAS DEL PROYECTO (GITHUB PROJECTS)



Cada barra va de la fecha de inicio más temprana a la de fin más tardía de las issues de la épica (campos Fecha inicio y Fecha fin del proyecto); la parte oscura es lo ya hecho.

ROADMAP · ÉPICAS DEL PROYECTO

DIAGRAMA DE GANTT · CRONOGRAMA DEL PROYECTO



Fechas de las fases según el cronograma de la memoria; el reparto por semanas del prototipo y del cierre es una previsión. #n = issue.

GANTT · OCTUBRE DE 2026 – ENERO DE 2027

06 Elección de tecnologías

Para que cualquier persona que lea esta memoria entienda las decisiones técnicas, conviene aclarar antes dos ideas: el **frontend** es la parte de la aplicación que el usuario ve y con la que interactúa en su navegador; el **backend** es la parte que funciona en un servidor, fuera de su vista, gestiona los datos y protege la información.

Frontend — Vue 3

Construye la interfaz visual. Sencillo de aprender, muy documentado, y las pantallas se actualizan solas cuando cambian los datos (el total del calendario se recalcula al añadir una receta, sin recargar la página).

Backend — Node.js + Express

Responde a las peticiones de la aplicación («dame mis recetas», «genera mi calendario»). Permite usar TypeScript tanto en frontend como en backend, simplificando el desarrollo.

TypeScript

Extensión de JavaScript que obliga a definir de antemano el tipo de cada dato. Evita errores típicos y el propio editor avisa antes de ejecutar nada.

Base de datos — PostgreSQL

Guarda de forma organizada y permanente usuarios, recetas, preferencias y calendarios. Gratuita, fiable y adecuada para datos relacionados entre sí.

Prisma (ORM)

Traductor entre el código y la base de datos: los datos se piden con código normal en vez de SQL, y avisa de errores de nombre o tipo antes de que el programa falle.

Seguridad — JWT y bcrypt

JWT es una «pulsera identificativa» digital tras iniciar sesión. bcrypt revuelve las contraseñas de forma irreversible antes de guardarlas en la base de datos.

Validación — Zod

Comprueba que los datos que llegan al servidor tienen el formato esperado antes de guardarlos, evitando datos incorrectos o malintencionados.

Pruebas — Vitest

Programas que comprueban automáticamente que el código sigue funcionando como se espera, detectando errores antes de que lleguen a producción.

Estado y navegación — Pinia y Vue Router

Pinia guarda en un único sitio los datos que necesita la aplicación (usuario, recetas, calendario) y Vue Router cambia de pantalla sin recargar la página, como en una aplicación de escritorio.

Componentes — Storybook

Botones, tarjetas y formularios se construyen como piezas reutilizables con prefijo propio (tp-), organizadas en átomos, moléculas y organismos. Storybook muestra cada pieza por separado, con su documentación y su código, para revisarla y probarla sin abrir toda la aplicación.

Despliegue — Vercel y Neon · Control de versiones — GitHub y GitHub Actions

Vercel aloja la parte visual y el servidor; Neon hace lo mismo con la base de datos: cada cambio subido se publica solo, sin servidor propio que mantener. GitHub guarda el historial de cambios y GitHub Actions construye y prueba el código automáticamente en cada cambio, avisando si algo se ha roto.

En resumen, estas tecnologías se han elegido para que el proyecto sea seguro, fácil de mantener y pueda publicarse en internet sin coste ni infraestructura propia, dejando el esfuerzo de desarrollo centrado en la parte más importante: el motor de reglas que genera los calendarios de comidas.

07 Diagrama de casos de uso

> Recoge qué puede hacer cada actor con Recetario. Los casos de uso se agrupan por bloque funcional y cada uno enlaza con una historia de usuario del backlog (US-xx).

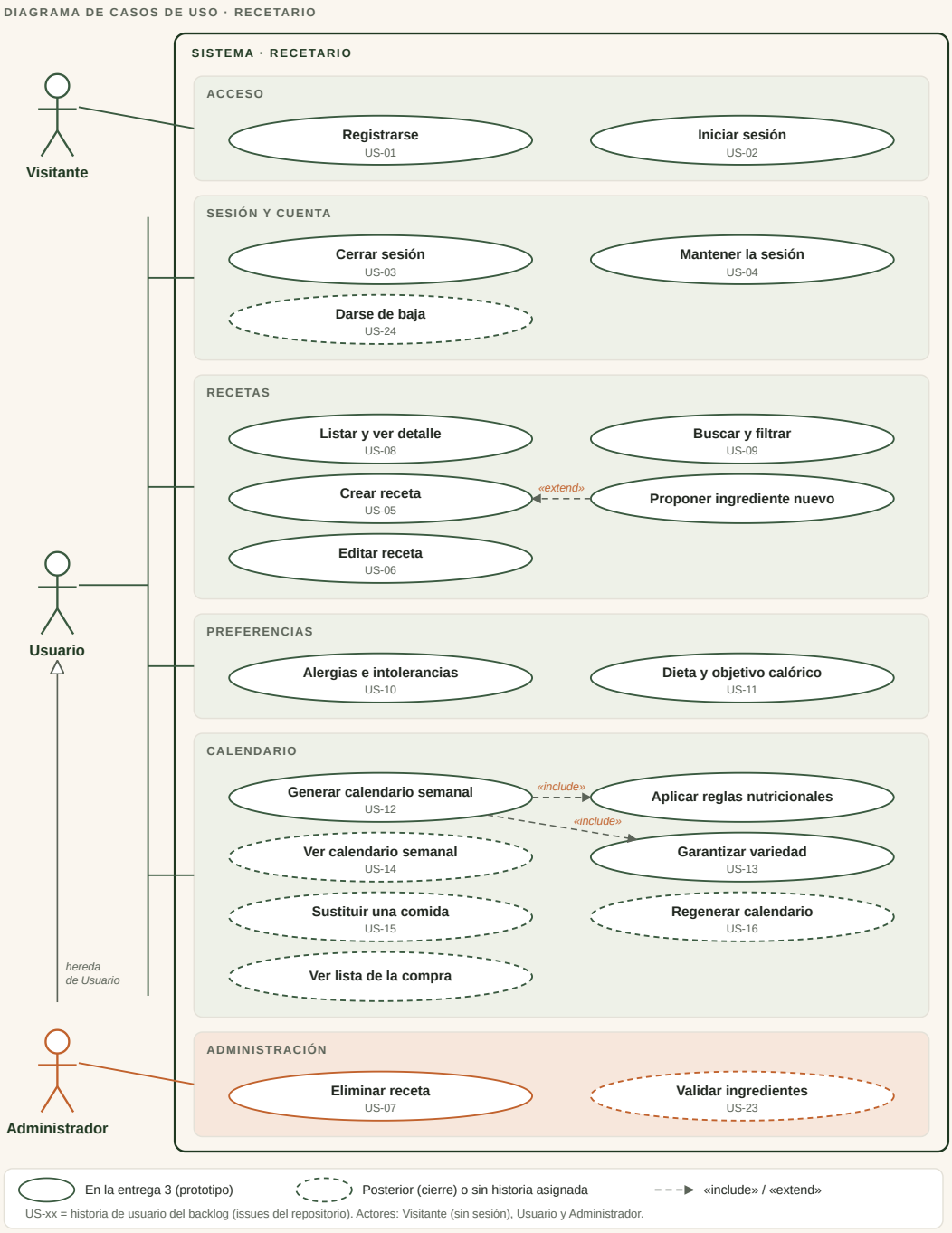


DIAGRAMA DE CASOS DE USO · 3 ACTORES

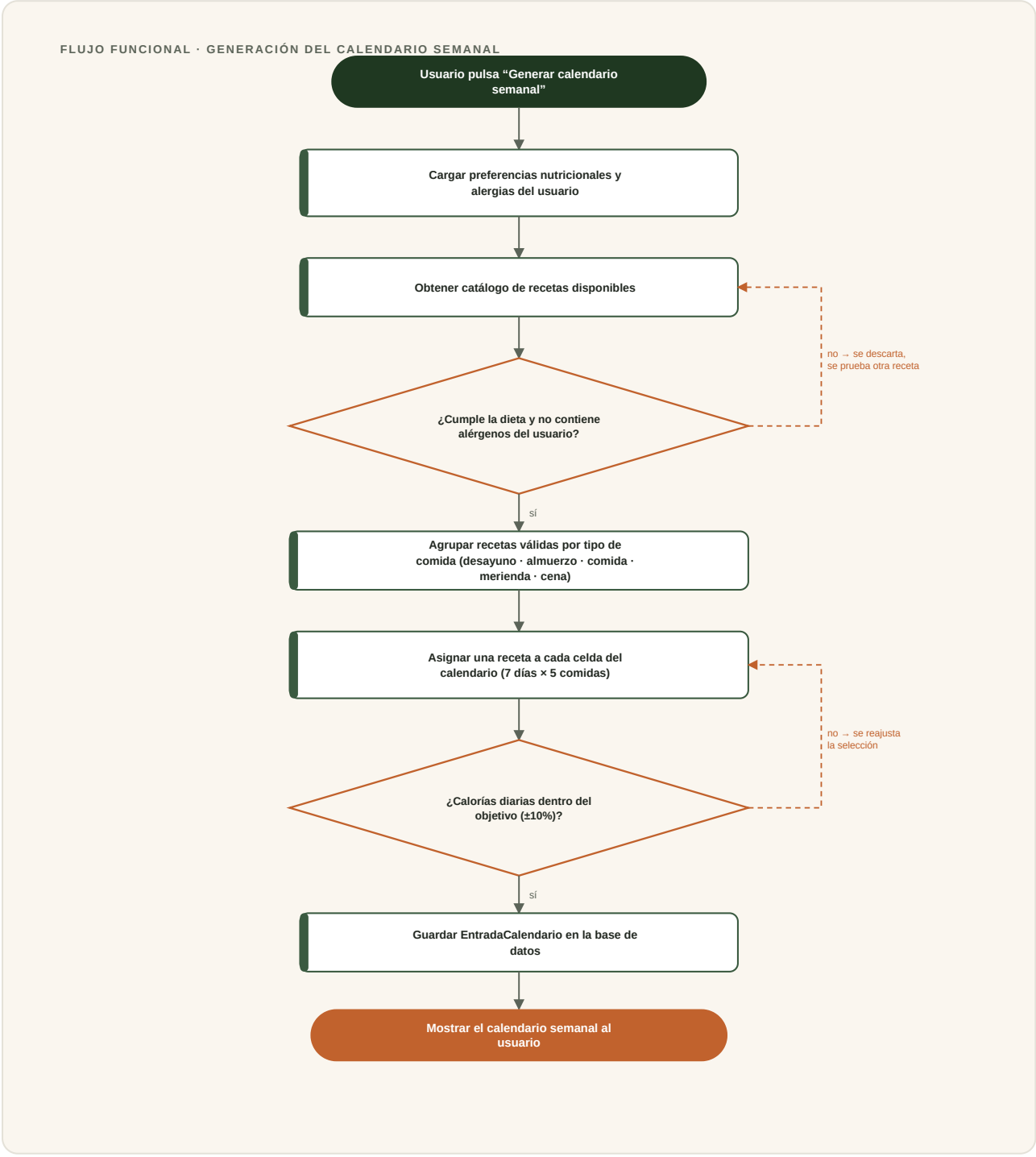
Visitante — sin sesión: solo puede registrarse o iniciar sesión.

Usuario — gestiona recetas, preferencias y su calendario semanal.

Administrador — hereda todo lo del usuario y además elimina recetas y valida ingredientes propuestos.

08 Diagrama de flujo funcional

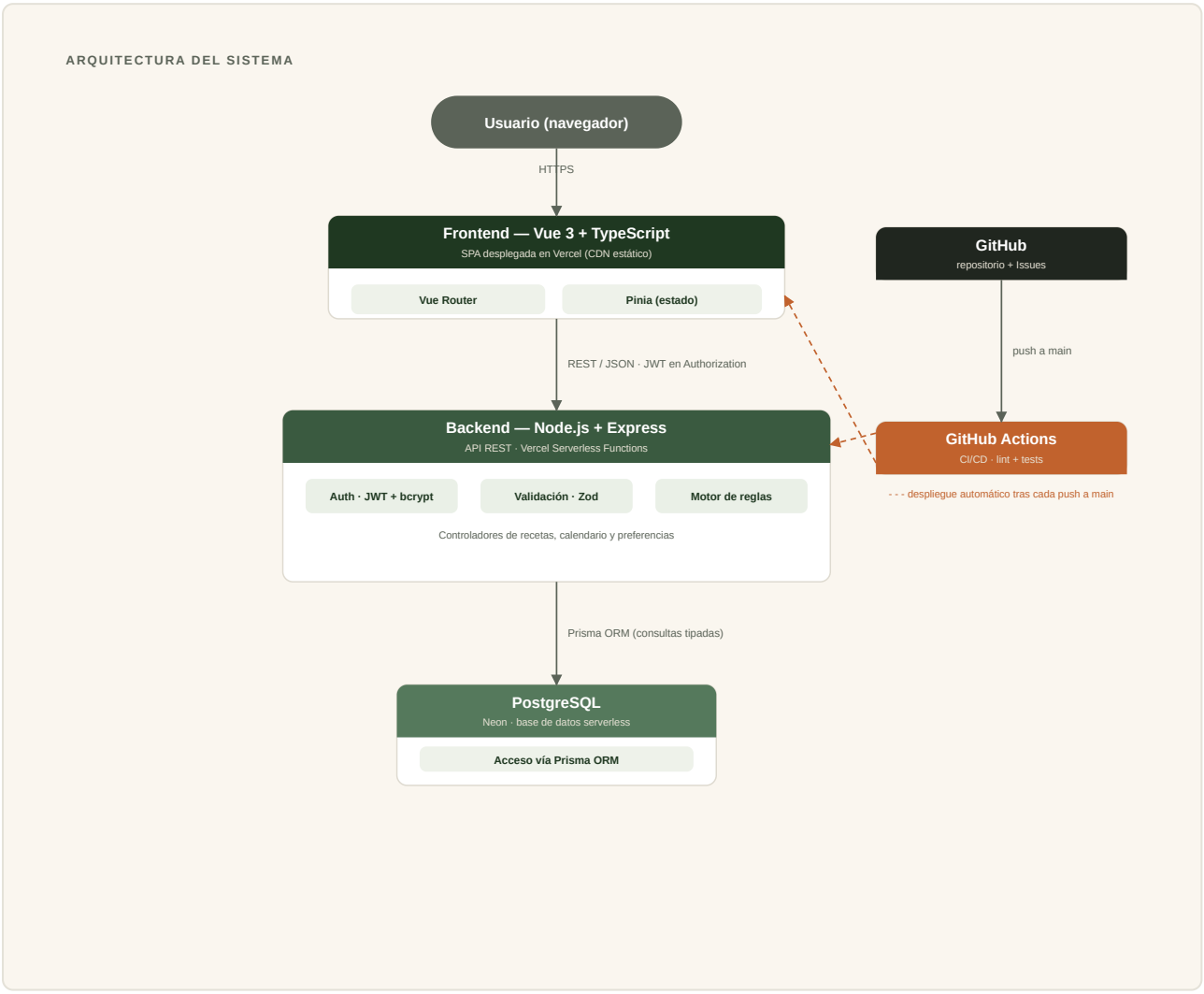
> Describe cómo el motor de reglas genera el calendario semanal: filtra recetas por dieta y alergias, las asigna a cada celda, y ajusta el resultado hasta cumplir el objetivo calórico.



FLUJO · GENERACIÓN AUTOMÁTICA DEL CALENDARIO SEMANAL

09 Diagrama de arquitectura

> Arquitectura en tres capas desacopladas, desplegadas por separado y conectadas mediante un pipeline de integración continua.



FRONTEND (VUE 3) · BACKEND (EXPRESS) · BASE DE DATOS (POSTGRESQL/NEON) · CI/CD (GITHUB ACTIONS)

El navegador consume una SPA en Vue 3 desplegada como contenido estático en Vercel. Esa SPA habla por HTTPS con una API REST en Node.js/Express (también en Vercel, como funciones serverless), que valida la identidad mediante JWT, los datos de entrada con Zod, y resuelve la lógica de negocio del motor de reglas. El acceso a PostgreSQL (alojado en Neon) se hace siempre a través de Prisma. Cada push a `main` dispara GitHub Actions, que ejecuta lint y tests antes de desplegar automáticamente frontend y backend.

10 Diagrama entidad-relación

> El modelo de datos se organiza en tres bloques: identidad y preferencias del usuario, catálogo de recetas, y planificación semanal.

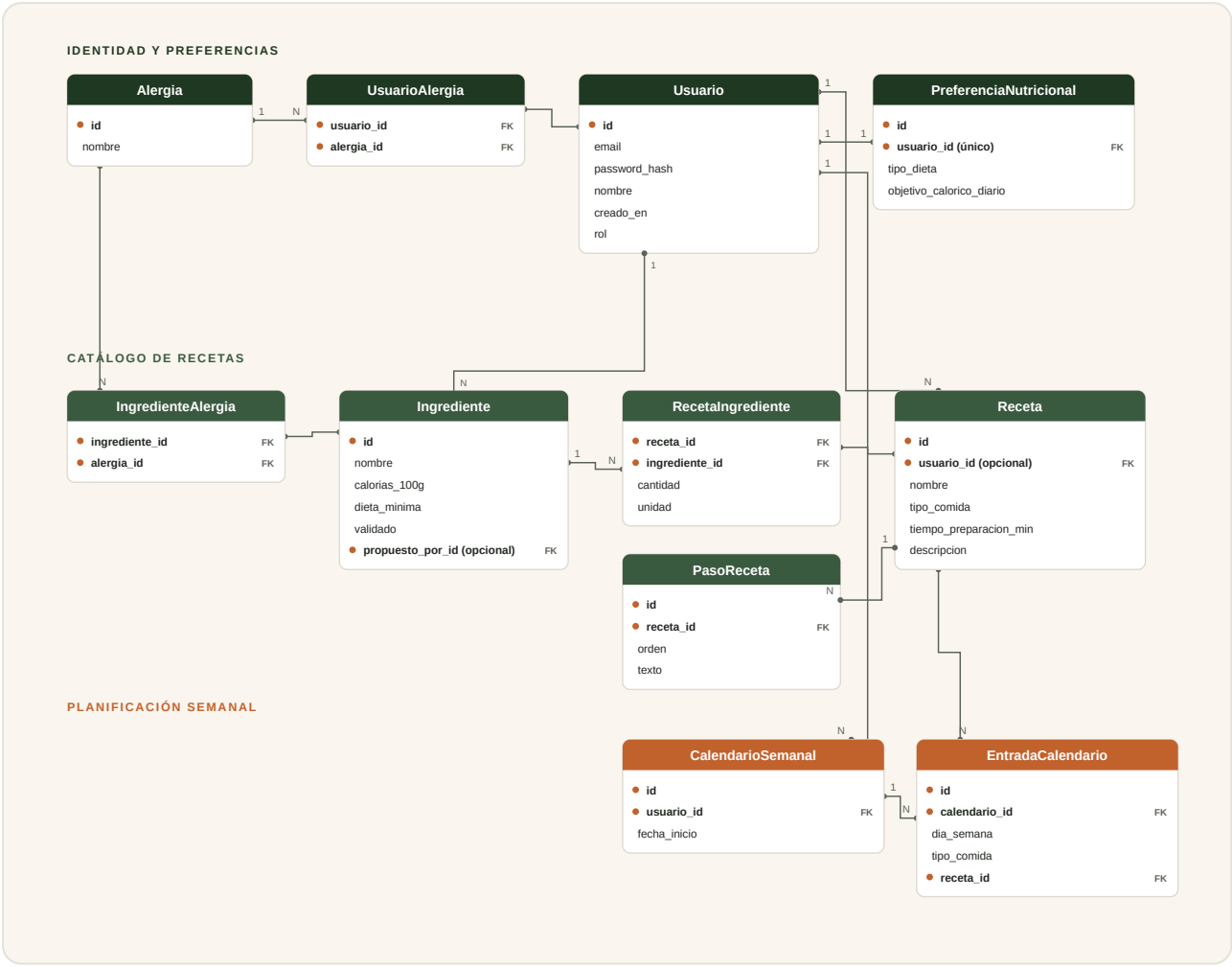


DIAGRAMA ENTIDAD-RELACIÓN · 11 ENTIDADES

Usuario — cuenta, autenticación y rol (usuario o administrador)

PreferenciaNutricional — dieta y objetivo calórico (1:1 con Usuario)

Alergia — catálogo de alergias/intolerancias

Ingrediente — catálogo con calorías por 100g, dieta mínima que lo admite y estado de validación (los que propone un usuario los valida el administrador)

Receta — pública para todos los usuarios; guarda su autor (se conserva aunque el autor se dé de baja), tipo de comida, tiempo y descripción

PasoReceta — pasos de preparación ordenados de cada receta

CalendarioSemanal — una semana por usuario

EntradaCalendario — celda día × tipo de comida

UsuarioAlergia / IngredienteAlergia — cruces N:M para el motor de reglas

RecetaIngrediente — cantidad y unidad por ingrediente

11 Esquema relacional

> Traducción del modelo entidad-relación a las 11 tablas de PostgreSQL que gestiona Prisma. Las tablas puente resuelven las relaciones N:M y las claves foráneas indican a qué tabla apuntan.

ESQUEMA RELACIONAL · POSTGRESQL (PRISMA)

IDENTIDAD Y PREFERENCIAS

usuario

PK	**id**	INT
	email	VARCHAR · UQ
	password_hash	VARCHAR
	nombre	VARCHAR
	rol	VARCHAR · «usuario»
	creado_en	TIMESTAMP

preferencia_nutricional

PK	**id**	INT
FK	usuario_id	INT · UQ → usuario.id
	tipo_dieta	VARCHAR
	objetivo_calorico_diario	INT

alergia

PK	**id**	INT
	nombre	VARCHAR · UQ

usuario_alergia

PF	**usuario_id**	INT → usuario.id
PF	**alergia_id**	INT → alergia.id

CATÁLOGO DE RECETAS

ingrediente

PK	**id**	INT
	nombre	VARCHAR · UQ
	calorias_100g	INT
	dieta_minima	VARCHAR · «Omnívora»
	validado	BOOLEAN · true
FK	propuesto_por_id	INT? → usuario.id

ingrediente_alergia

PF	**ingrediente_id**	INT → ingrediente.id
PF	**alergia_id**	INT → alergia.id

receta

PK	**id**	INT
FK	usuario_id	INT? → usuario.id
	nombre	VARCHAR
	tipo_comida	VARCHAR
	tiempo_preparacion_min	INT
	descripcion	VARCHAR

receta_ingrediente

PF	**receta_id**	INT → receta.id
PF	**ingrediente_id**	INT → ingrediente.id
	cantidad	DOUBLE
	unidad	VARCHAR

paso_receta

PK	**id**	INT
FK	receta_id	INT → receta.id
	orden	INT · UQ(receta, orden)
	texto	VARCHAR

PLANIFICACIÓN SEMANAL

calendario_semanal

PK	**id**	INT
FK	usuario_id	INT → usuario.id
	fecha_inicio	TIMESTAMP

entrada_calendario

PK	**id**	INT
FK	calendario_id	INT → calendario_semanal.id
	dia_semana	VARCHAR
	tipo_comida	VARCHAR
FK	receta_id	INT? → receta.id

UQ (calendario_id, dia_semana, tipo_comida)

LEYENDA Y BORRADO

PK Clave primaria
FK Clave foránea
PF PK compuesta formada por FK

tipo? = columna opcional (NULL)

Al borrar el registro padre

- CASCADE: tablas puente, pasos, ingredientes de receta, calendarios y preferencias.
- SET NULL: autor de receta y receta de entrada.

ESQUEMA RELACIONAL · 11 TABLAS

Lista de la compra — no es una tabla: se calcula sumando los ingredientes de las recetas del calendario.

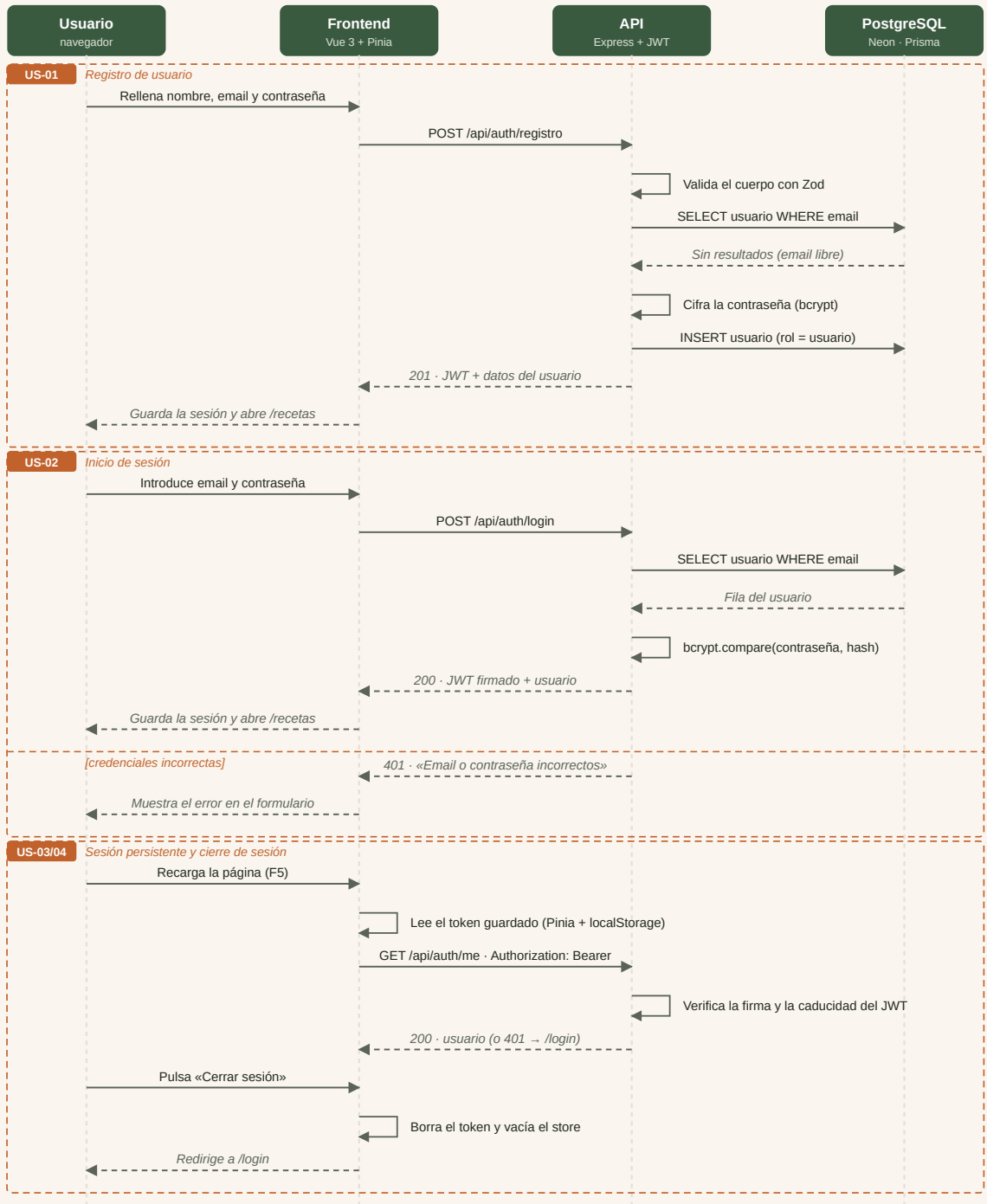
Recetas públicas — el autor es opcional (SET NULL) para conservar la receta si el usuario se da de baja.

Ingredientes — los propone un usuario y los valida el administrador (validado, propuesto_por_id).

12 Secuencia: autenticación

> Registro, inicio de sesión, sesión persistente y cierre de sesión (US-01 a US-04). Es el diseño previsto: los nombres exactos de los endpoints se fijarán al implementar cada historia.

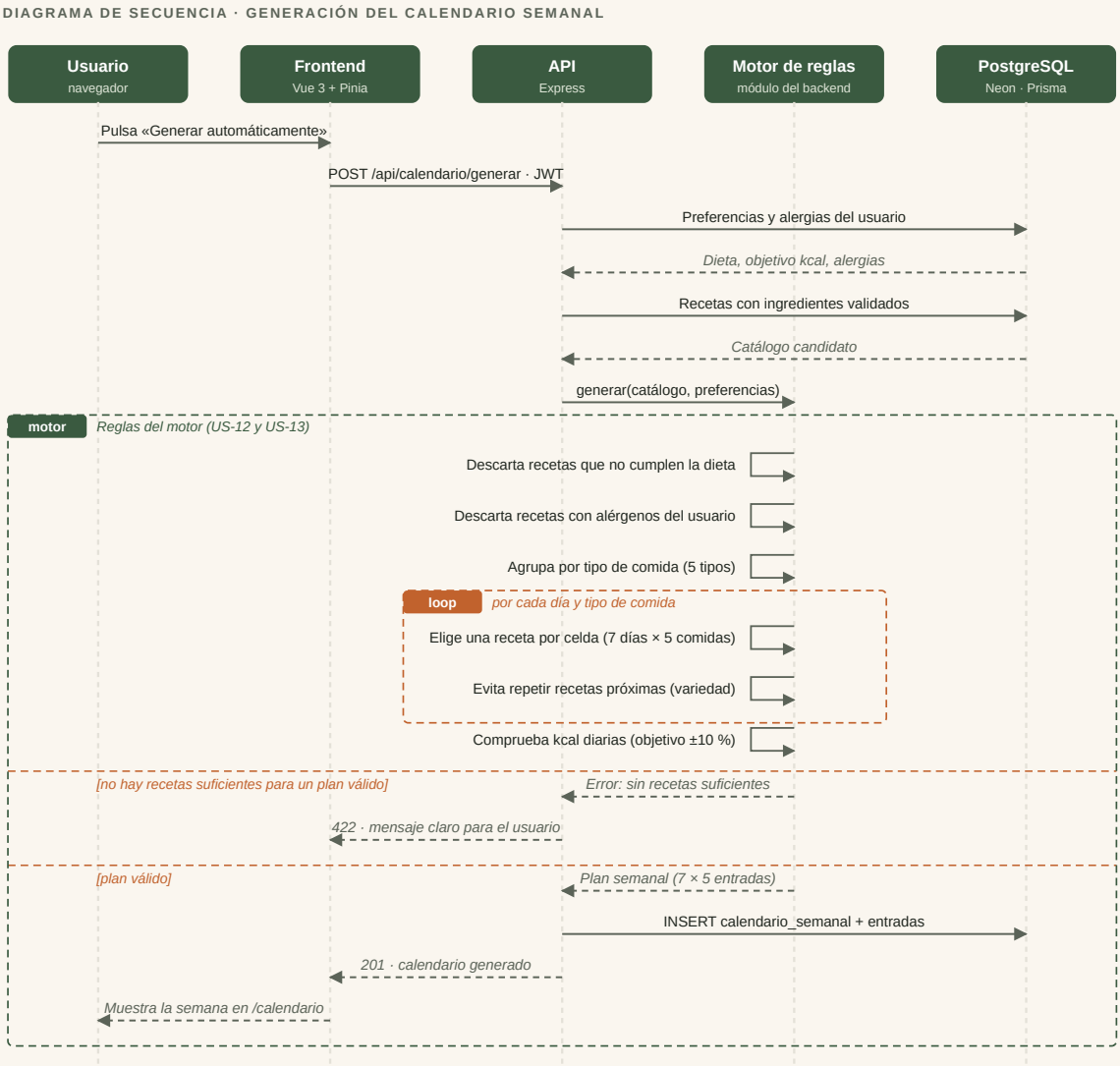
DIAGRAMA DE SECUENCIA · AUTENTICACIÓN (REGISTRO, LOGIN Y SESIÓN)



SECUENCIA · REGISTRO, LOGIN Y SESIÓN CON JWT

13 Secuencia: generación del calendario

> Cómo genera el motor de reglas el calendario semanal (US-12 y US-13): filtra por dieta y alergias, asigna una receta a cada celda evitando repeticiones y comprueba el objetivo calórico.



SECUENCIA · GENERACIÓN AUTOMÁTICA DEL CALENDARIO

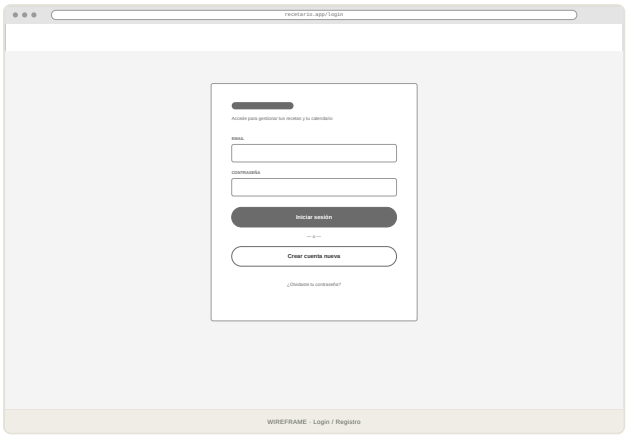
Entrada — preferencias, alergias y recetas con ingredientes validados.

Salida — 7 días × 5 comidas guardadas en entrada_calendario.

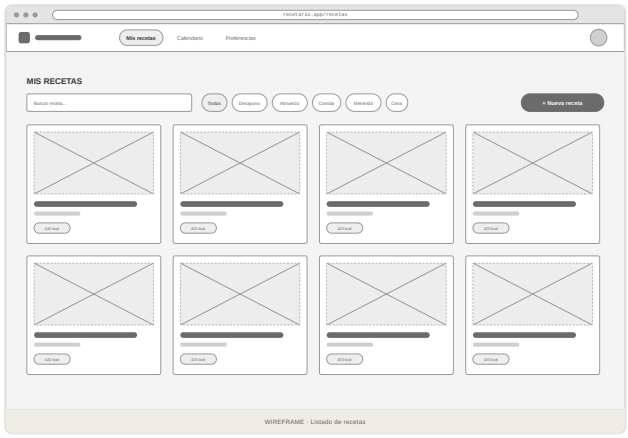
Fallo — si no hay recetas suficientes, se informa con un mensaje claro.

14 Mockups de las pantallas principales

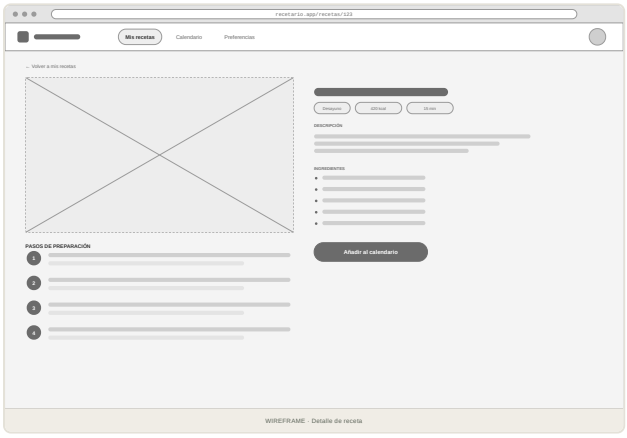
> Wireframes de baja fidelidad: fijan la estructura y el contenido de cada pantalla sin entrar todavía en el diseño visual, que se abordará en una fase posterior.



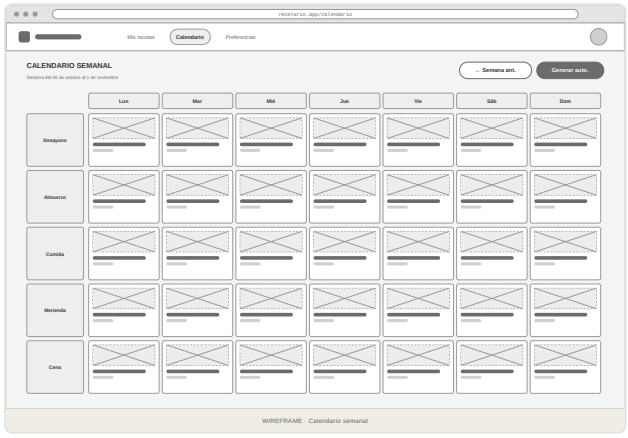
LOGIN / REGISTRO



LISTADO DE RECETAS



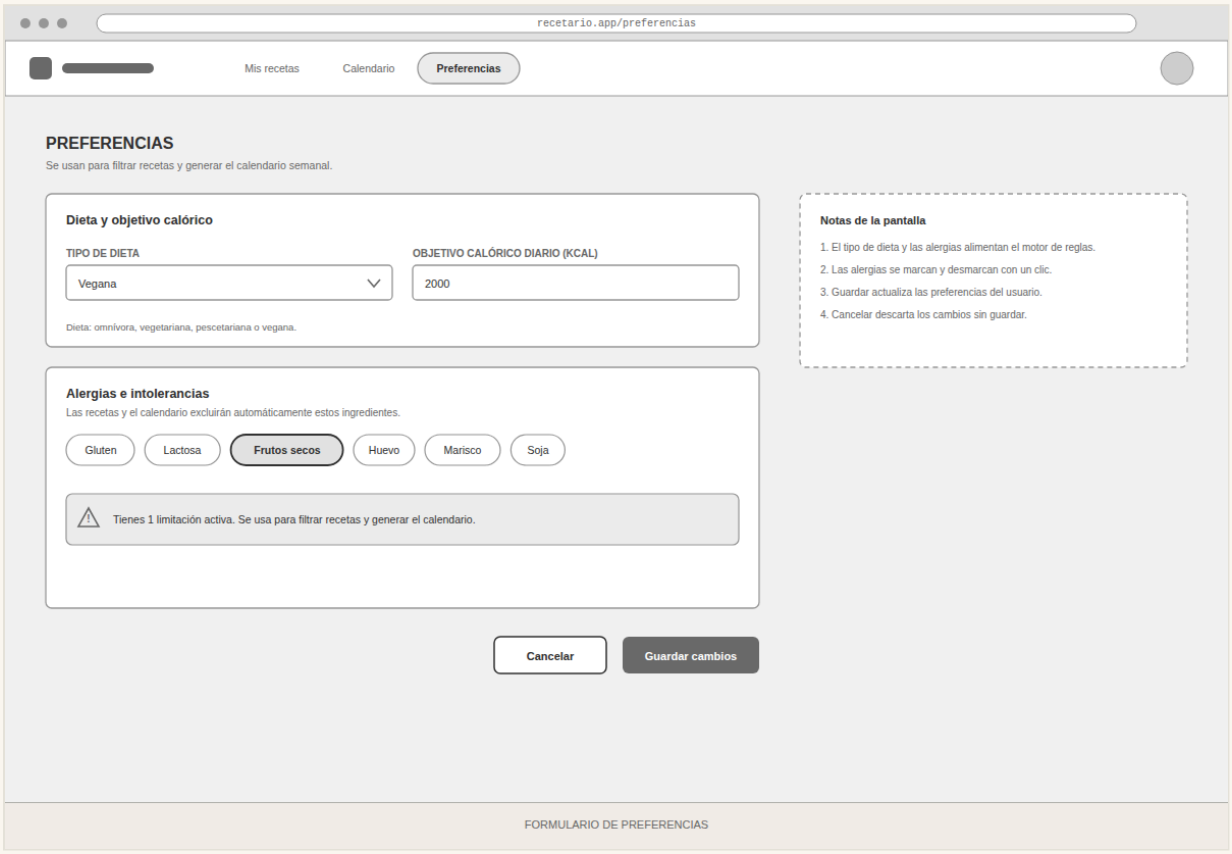
DETALLE DE RECETA



CALENDARIO SEMANAL

14 Mockups (continuación)

Wireframe del formulario de preferencias, la quinta pantalla clave: recoge el tipo de dieta, el objetivo calórico diario y las alergias e intolerancias, datos que alimentan el motor de reglas al generar el calendario semanal.



FORMULARIO DE PREFERENCIAS

15 Control de versiones y estado del esqueleto

El código se versiona con Git y se aloja en un repositorio público de GitHub. Al ser un desarrollo individual, el flujo de trabajo es sencillo: cada historia de usuario se desarrolla en su propia rama (`feature/us-XX-descripcion`), con commits pequeños y descriptivos, y se integra en `main` mediante un pull request, lo que deja un historial organizado y revisable aunque no haya otra persona revisando.

CI automática

GitHub Actions construye y prueba backend y frontend en cada pull request hacia `main` y en cada subida a `main`.

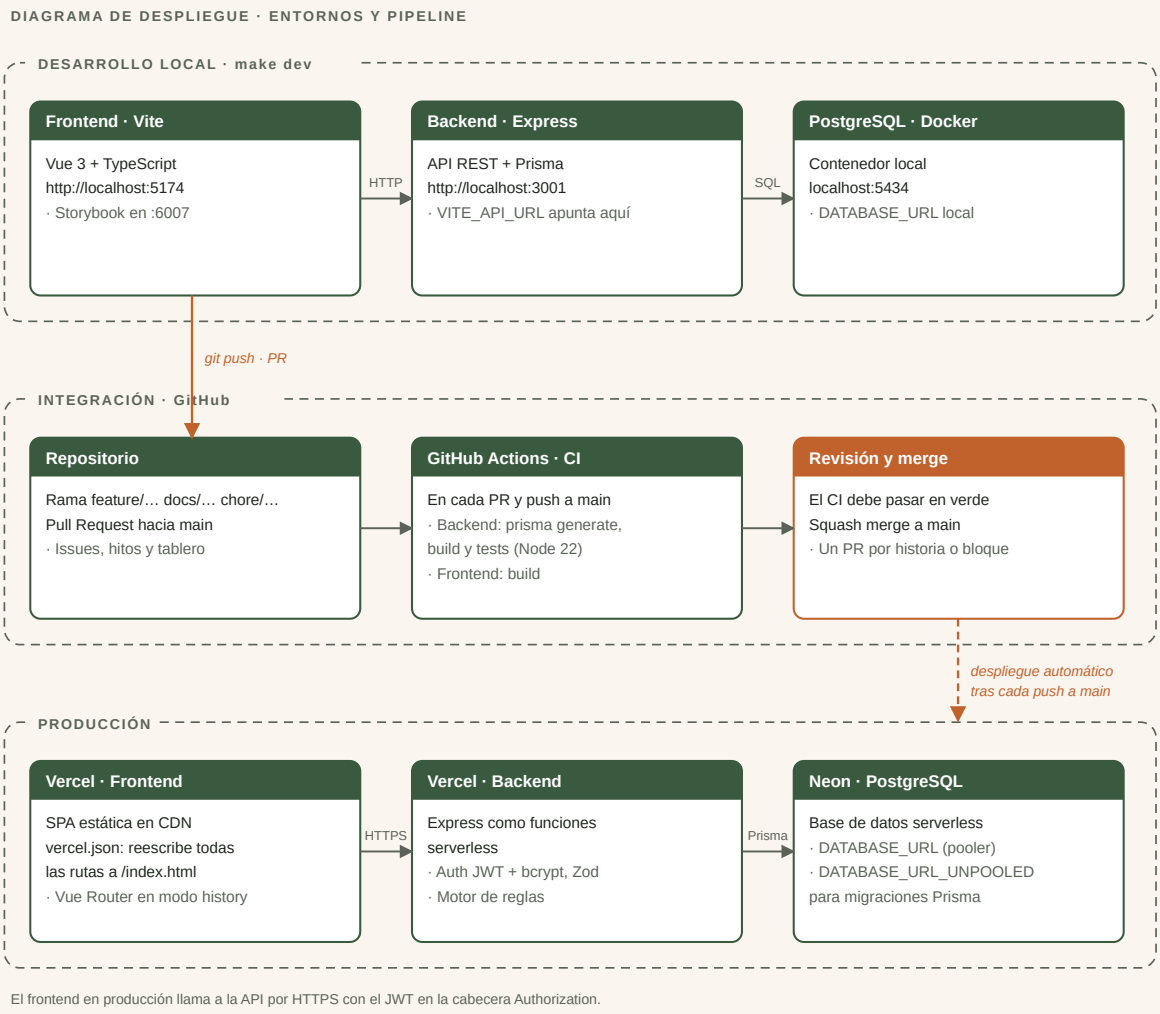
Tablero Kanban

Backlog → Lista para hacer → En progreso → En revisión/test → Hecho.

Esqueleto desplegado. El proyecto Vue 3 y el proyecto Express están conectados entre sí y publicados en Vercel, con PostgreSQL en Neon y las migraciones aplicadas; la página de inicio comprueba la conexión con la API. El desarrollo real (autenticación, CRUD de recetas, motor de reglas) arranca en la fase «Prototipo (~50%)», del 9 de noviembre al 6 de diciembre.

16 Diagrama de despliegue

> Entornos del proyecto: desarrollo local con make dev, integración con GitHub y Actions, y producción en Vercel y Neon. Complementa el diagrama de arquitectura.



DESPLIEGUE · ENTORNOS Y PIPELINE